SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

(SUBJECT: ARTIFICIAL NEURAL NETWORK)

(SUBJECTCODE:24SBT113)

A Individual Task on

"TRAIN A PRECEPTRON ON AND/OR TASKS"

Submitted in the partial fulfillment of the requirements for the fourth semester

BACHELOR OF TECHNOLOGY IN AIML

Submitted By,

SNEHA S(01SU24AI100)

UNDER THE GUIDANCE OF

Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2025-26

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

Department of ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that SNEHA S (01SU24AI100) has satisfactorily completed the assessment (Individual-Task – Module 3) in **"ARTIFICIAL NEURAL NETWORK "** prescribed by the Srinivas University for the 4st semester B. Tech course during the year **2025-26**.

MARKS AWARDED

Staff In charge

Name: Prof. Mahesh Kumar V B

Assistant Professor, Department Of AIML

TABLE OF CONTENTS

SI.no	Description	Page no.
1	Introduction	1
2	Objectives	1
3	Theoretical Background	2
4	Dataset Description	2-3
5	Methodology	3-4
6	Error-Correction Learning on AND Task	5
7	Error-Correction Learning on OR Task	5
8	Effect of Learning Rate on Convergence	5-6
9	Comparision of AND vs OR Learning	6
10	Results and Discussion	6
11	Advantages of Error-Correction Learning	7
12	Limitations	7
13	Applications	7
14	Conclusion	8
15	Future Scope	8
16	References	9

Abstract

Error-correction learning is a fundamental supervised learning mechanism used in artificial neural networks to improve prediction accuracy by minimizing the difference between predicted and target outputs. This project demonstrates the working of the perceptron learning algorithm using AND and OR logical operations. The perceptron is trained using the error-correction learning rule, and the decrease in classification error is observed across iterations. Additionally, the effect of different learning rates on convergence speed and stability is analyzed. The experiment highlights how weight adjustments gradually reduce error and lead to correct classification, providing practical insight into the learning behavior of simple neural networks.

1. Introduction

Artificial Neural Networks (ANNs) learn patterns from data through iterative adjustments of weights. One of the most basic and important learning mechanisms is **error-correction learning**, where the model updates its parameters based on the difference between actual and predicted outputs.

The **Perceptron** is a simple neural network that uses this learning approach to solve binary classification problems. Logical operations such as AND and OR are classic examples used to demonstrate perceptron learning because they are linearly separable and easy to analyze.

This project focuses on:

- Training a perceptron on AND and OR datasets
- Observing error reduction during training
- Studying the influence of learning rate on convergence

2. Objectives

The objectives of this project are:

1. To understand error-correction learning in neural networks
2. To train a perceptron using AND and OR datasets
3. To analyze how error decreases over iterations
4. To study the effect of learning rate on convergence
5. To compare learning behavior between the two tasks

3. Theoretical Background

3.1 Error-Correction Learning

Error-correction learning adjusts weights in proportion to the error:

$$Error = Target - Output$$

The perceptron updates weights to reduce this error, moving the decision boundary toward correct classification.

3.2 Perceptron Model

The perceptron computes a weighted sum:

$$z = \sum w_i x_i + b$$

Output is obtained using a step activation function:

$$y = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

3.3 Weight Update Rule

$$w_i^{new} = w_i^{old} + \eta(t - y)x_i$$
$$b^{new} = b^{old} + \eta(t - y)$$

Where $\eta$is the learning rate.

4. Dataset Description

AND Dataset

x1	x2	Target
0	0	0
0	1	0
1	0	0

x1	x2	Target
1	1	1

OR Dataset

x1	x2	Target
0	0	0
0	1	1
1	0	1
1	1	1

Both datasets are linearly separable, making them suitable for perceptron learning.

5. Methodology

Training Procedure

The perceptron is trained using an iterative error-correction learning process. During training, the model repeatedly processes the dataset and updates its parameters until the classification error is minimized or becomes zero. The steps involved in the training procedure are described below.

5.1. Initialize Weights and Bias

At the beginning of training, the weights ($w_1, w_2, ...$) and bias (b) are assigned small initial values, often zero or random numbers. These parameters determine the initial position of the decision boundary. The learning rate (η) is also chosen at this stage, which controls how large each weight update will be.

5.2. Compute the Weighted Sum

For each input sample, the perceptron calculates the net input by taking the weighted sum of all input features and adding the bias term.

$$z = \sum w_i x_i + b$$

This value represents how strongly the input belongs to a particular class and indicates its position relative to the decision boundary.

5.3. Apply Activation Function

The net input is passed through a step activation function to produce the predicted output. The function converts the continuous value of z into a discrete class label (0 or 1), enabling binary classification.

5.4. Calculate Error

The predicted output is compared with the actual target value. The difference between them is the error, which indicates whether the prediction is correct.

$$Error = t - y$$

If the error is zero, the prediction is correct and no update is required. Otherwise, the perceptron adjusts its parameters.

5.5. Update Weights and Bias

Using the error-correction learning rule, the weights and bias are updated. The update moves the decision boundary in a direction that reduces the classification error for the misclassified sample. The magnitude of the update depends on the learning rate and input values.

5.6. Repeat for Multiple Epochs

Steps 2 to 5 are repeated for every sample in the dataset. Completing one full pass through all training samples is called an **epoch**. The training process continues for multiple epochs until the perceptron correctly classifies all samples or reaches a predefined stopping condition.

Recording Total Error

At the end of each epoch, the total error (sum of individual sample errors) is calculated. Tracking this value helps visualize the learning progress. As training proceeds, the total error typically decreases and eventually becomes zero, indicating that the perceptron has converged and learned the correct decision boundary.

6. Error-Correction Learning on AND Task

Observations

- Initially, the perceptron makes several incorrect predictions because the weights and bias are not yet optimized for the dataset.
- After each epoch, the weight updates shift the decision boundary, reducing the number of misclassified samples.
- As training continues, the model becomes more accurate and the classification error gradually approaches zero, indicating improved learning performance.

Error Trend

The total error shows a decreasing pattern over successive epochs. As the perceptron learns the correct decision boundary, fewer updates are needed, and the error reduces steadily. This monotonic decrease in error demonstrates the effectiveness of the error-correction learning rule in guiding the model toward convergence.

7. Error-Correction Learning on OR Task

Observations

- The OR task converges faster than the AND task because most input combinations belong to Class 1, making it easier for the perceptron to learn the correct decision boundary.
- Since fewer samples are misclassified during training, the number of weight updates required is comparatively lower.
- As a result, the model stabilizes quickly, reaching a point where predictions remain consistent across epochs.

Error Trend

The total error decreases rapidly during the initial epochs as the perceptron quickly adjusts its weights to match the target outputs. After a few iterations, the error reaches zero, indicating that the perceptron has successfully learned the OR function and achieved convergence.

8. Effect of Learning Rate on Convergence

The learning rate determines the step size of weight updates.

Small Learning Rate

- Slow convergence
- Stable learning

- More epochs required

Moderate Learning Rate

- Faster convergence
- Balanced stability

Large Learning Rate

- Faster but may oscillate
- Risk of overshooting

9. Comparison of AND vs OR Learning

Aspect	AND	OR
Convergence speed	Moderate	Fast
Number of updates	More	Less
Error reduction pattern	Gradual	Rapid

This shows how dataset structure influences learning dynamics.

10. Results and Discussion

The experiment confirms that:

- **Error-correction learning effectively trains the perceptron**, as the model continuously adjusts its weights based on the difference between predicted and target outputs.
- **Error decreases with each iteration**, showing that the learning rule guides the perceptron toward more accurate classifications over time.
- **Convergence depends on the learning rate**, where an appropriate value ensures steady and stable learning, while extreme values may slow down or destabilize training.
- **Linearly separable problems are solved successfully**, demonstrating that the perceptron can form a clear decision boundary when such a boundary exists.

Overall, the results highlight the importance of selecting a suitable learning rate to achieve a balance between convergence speed and stability, ensuring efficient training without oscillations.

.

11. Advantages of Error-Correction Learning

- **Simple and intuitive learning mechanism:** The learning process is straightforward, as the model adjusts its weights directly based on the observed error, making it easy to understand conceptually.
- **Ensures gradual improvement:** With each iteration, the perceptron reduces misclassification, leading to steady progress toward accurate predictions.
- **Easy to implement:** The algorithm requires only basic mathematical operations, allowing it to be implemented manually, in spreadsheets, or in simple programs.
- **Works well for linear problems:** Error-correction learning is effective when the dataset is linearly separable, enabling the model to find a clear decision boundary between classes.

12. Limitations

- **Not suitable for non-linear datasets:** The perceptron with error-correction learning can only form a linear decision boundary, so it cannot correctly classify problems where the classes are not linearly separable.
- **Sensitive to parameter choice:** The performance of the learning process depends on parameters such as the learning rate; inappropriate values can lead to slow convergence or unstable updates.
- **May require many iterations:** For certain datasets, especially when initial weights are far from optimal, the model may need multiple epochs before reaching convergence.

13. Applications

- **Binary classification problems**
  Error-correction learning is widely used in simple classification tasks where data needs to be separated into two categories, such as yes/no or true/false decisions.
- **Pattern recognition**
  It can be applied to recognize basic patterns in data, including simple image or signal features, by learning to distinguish between different classes.
- **Basic control systems**
  In control applications, perceptron-based learning can help systems make decisions based on input conditions, such as switching mechanisms or threshold-based responses.
- **Signal detection**
  The learning rule can be used to identify the presence or absence of a signal in noisy environments by classifying input patterns accordingly.

14. Conclusion

This project demonstrated the concept of error-correction learning using a perceptron trained on AND and OR logical tasks. Through systematic training and observation of weight updates, the experiment clearly illustrated how classification error decreases over successive iterations. As the perceptron adjusted its weights based on the difference between predicted and target outputs, it gradually improved its performance and successfully converged for both tasks. This confirms the effectiveness of the perceptron learning rule for solving linearly separable problems.

The analysis of the learning rate highlighted its important role in influencing convergence behavior. A moderate learning rate enabled steady and efficient learning, allowing the model to reach stability in fewer epochs, while very small or large values either slowed the process or caused fluctuations. This demonstrates the need for careful parameter selection in neural network training.

Overall, the study provides a clear and practical understanding of how neural networks learn from errors and refine their decision boundaries through iterative updates. The insights gained from this experiment form a strong conceptual foundation for exploring more advanced learning algorithms and optimization methods used in modern machine learning and deep learning systems.

15. Future Scope

- **Implement experiment in Python**
  Automating the perceptron training process in Python would allow faster experimentation, easier weight updates, and calculation of performance metrics such as accuracy and loss.
- **Plot error vs epoch graphs**
  Visualizing how the total error changes across epochs can provide a clearer understanding of the learning behavior and convergence pattern of the model.
- **Extend to non-linear datasets**
  Applying the experiment to more complex datasets and using advanced models like multilayer perceptrons can help overcome the limitations of linear separability.
- **Compare with other learning rules**
  Studying alternative learning approaches such as Hebbian learning or reinforcement learning would provide deeper insight into different training strategies and their effectiveness.

16. References

1. Rosenblatt, F. — Perceptron Theory
2. Simon Haykin — Neural Networks
3. Goodfellow — Deep Learning
4. Neural Network lecture notes